Student Submission Template

Experiment 8: Modular Design of a Software System

Student / Group	Details
Name(s)	MS Suhan Gaurav Vyas Shubham Parmar Kunal Roy
Roll Number(s)	20252651036 20252651020 20252651054 20252651031
Project Title	Shield AI — Scam Detection System
Date	28 October 2026

1. Aim and project overview

Aim: Divide our own software project into meaningful modules, define interfaces, and justify cohesion and coupling.

What does your software do? (3–4 lines):

Shield AI accepts text copied from email, SMS, chat or other sources and analyses it for possible scam indicators. It checks message wording and URLs where possible, then returns a risk verdict and an explanation. Scan history and feedback can be supported as optional features.

2. Main functional requirements

No.	Requirement / action performed by your project
1	Accept and validate submitted text
2	Preprocess text and extract URLs
3	Detect suspicious language and scam patterns
4	Check URL/domain reputation when available
5	Combine signals and generate verdict/explanation
6	Display results to the user
7	Optionally save scan history
8	Optionally collect feedback

3. Module identification

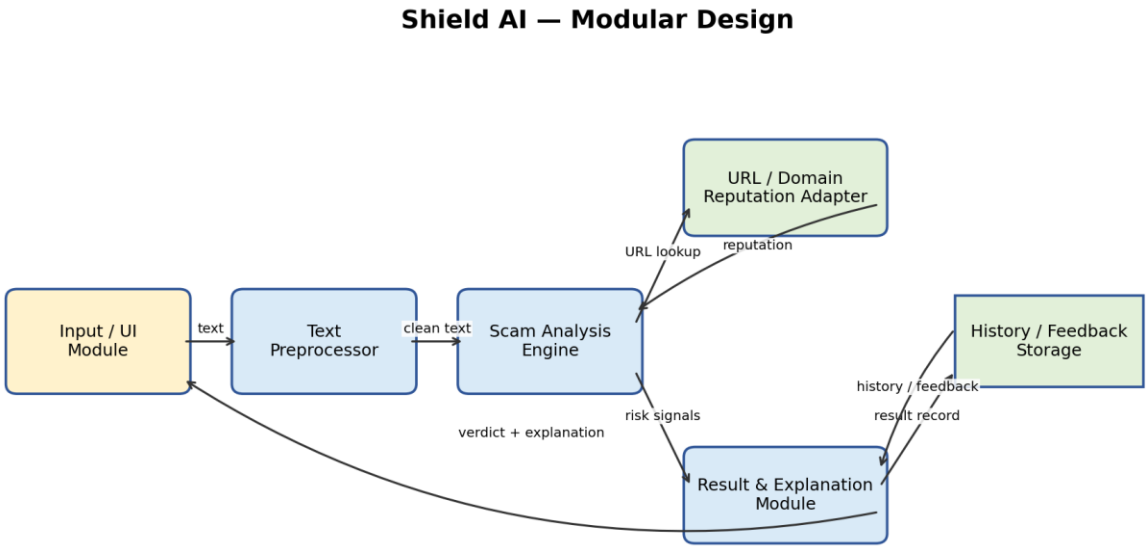
Create modules based on YOUR project, not automatically the Library Management example.

Module name	Main responsibility	Requirements covered
Input / UI Module	Accepts text and presents the scan result.	FR-01, FR-02, FR-07
Text Preprocessing Module	Normalises input and extracts URLs/features.	FR-03, FR-05
Scam Analysis Engine	Evaluates suspicious language, urgency and requests for secrets.	FR-04
URL Reputation Adapter	Queries configured reputation service and normalises response.	FR-05, FR-06
Result & Explanation Module	Combines signals and formats verdict, score and reasons.	FR-07
History / Feedback Module	Optionally stores scan metadata and	FR-08

	user feedback.	
Configuration Module	Provides rules, thresholds and service settings.	FR-04, FR-06

4. Modular design diagram

Draw boxes for each module. Use labeled arrows to show which module requests services from another module. You may paste a diagram below.



5. Interfaces between modules

Example only: Loan → Book Catalogue; checkAvailability(bookId) returns yes/no.

Calling module → Providing module	Operation / interface	Input	Output
Input/UI → Preprocessing	preprocessText(text)	Raw message text	Clean text and extracted URLs
Analysis Engine → URL Adapter	check_url(url)	URL/domain string	Risk result or lookup-unavailable status
Analysis Engine → Result Module	buildResult(signals)	Structured risk signals and score	Verdict and explanation
Result / Feedback → History Store	saveScan(record) / saveFeedback(data)	Scan metadata or feedback label	Record ID / success status

6. Cohesion: why do these tasks belong together?

Module	Tasks inside it	Why this has high cohesion
Text Preprocessing	Normalisation, cleanup and URL extraction	All tasks transform raw input into consistent analysis-ready text/features.
Scam Analysis Engine	Phrase, urgency, impersonation and sensitive-data request checks	All tasks assess scam indicators and produce structured signals.
Result & Explanation	Risk verdict, explanation and formatting	All tasks convert the same assessment into clear, understandable output.

7. Coupling: how did you reduce dependencies?

Module connection	Needed? Why?	How to keep coupling low
UI → Analysis Engine	Yes; the UI must request an analysis	Use a stable analyse(text) interface; UI does not implement detection rules.
Analysis Engine → URL Adapter	Conditional; only when a URL is present and lookup is configured	Hide external provider details behind an adapter and handle unavailable service.
Result Module → History / Feedback	Optional; only if persistence is enabled	Use a repository interface and avoid making analysis depend on a specific database.

8. Design review checklist

☐ Every important requirement belongs to at least one module.

☐ Every module has a clear purpose.

☐ Diagram matches the module table.

☐ Interfaces show inputs and outputs.

☐ Unnecessary direct dependencies have been avoided.

9. Result (write in your own words)

In this experiment, we designed the modules of Shield AI. We identified seven modules and defined interfaces between them. We improved cohesion by grouping related tasks and reduced coupling by using clear module interfaces and an adapter for external URL reputation services.